



ACIF104 Aprendizaje de máquina

Nombres:

JUAN CONDORI

NELSON CARRASCO ZEPEDA

RUBEN DELGADO ARRIAGADA

SEBASTIAN NUÑEZ SOTO

Curso: Ingeniería Civil Informática

Fecha: 07/07/2026

1. **Configuración del Entorno de Desarrollo y Viabilidad de Hardware**

Para garantizar la reproducibilidad y el aislamiento de las dependencias del proyecto, se habilitó un entorno de desarrollo basado en las siguientes especificaciones técnicas:

- **Entorno Virtual y Herramientas Computacionales:** Se configuró un entorno aislado (.venv) utilizando Python en su versión estable. Como Entorno de Desarrollo Integrado (IDE) se seleccionó Visual Studio Code, complementado con extensiones de depuración interactiva.
- **Bibliotecas del Sistema:** El ecosistema de librerías esenciales incluye SQLAlchemy y pyodbc para la gestión de la conexión y extracción del Data Warehouse; Pandas y NumPy para la manipulación matricial de alta eficiencia; Scikit-Learn para las particiones estadísticas y reportes de rendimiento; Imbalanced-Learn para la inyección del algoritmo SMOTE; y Joblib para la persistencia del modelo en disco duro. Para el subsistema gráfico se implementaron Matplotlib y Seaborn.
- **Optimización de Hardware:** Debido a la carga computacional que representa procesar un dataset expandido por codificación binaria a 123 columnas y balanceado sintéticamente a nivel de entrenamiento, se parametrizó el algoritmo utilizando el argumento `n_jobs=-1`. Esta directiva técnica instruye al compilador a distribuir la carga en paralelo a través de todos los hilos físicos y lógicos de la Unidad Central de Procesamiento (CPU) disponibles en el hardware local, evitando cuellos de botella y optimizando el tiempo de

2. Especificación y Justificación de la Arquitectura del Modelo

Tras evaluar las limitaciones en redes neuronales densas (MLP) y arquitecturas tabulares complejas (TabNet) frente a la alta dispersión de los datos y las restricciones de hardware, se seleccionó el modelo XGBoost (Extreme Gradient Boosting) como la arquitectura predictiva definitiva para el sistema bautizado como "Finan".

- **Estructura de Entradas y Salidas:** La capa de entrada del modelo recibe un vector disperso de 123 características continuas y binarias (derivadas del preprocesamiento estadístico). La capa de salida consiste en un único nodo sigmoidal que mapea una distribución probabilística binaria: Clase 0 (Transacción Legítima) y Clase 1 (Transacción Fraudulenta).
- **Parámetros de la Arquitectura y Convergencia:** El algoritmo se estructuró como un ensamble secuencial de 100 estimadores (árboles de decisión base) con una profundidad máxima (max_depth) acotada a 5 niveles para prevenir el sobreajuste y limitar la complejidad estructural. La tasa de aprendizaje (learning_rate) se fijó en 0.1 para asegurar un descenso de gradiente suave pero constante. Como función de pérdida para guiar el aprendizaje en cada iteración, se definió la entropía cruzada binaria (eval_metric='logloss'), idónea para problemas de clasificación excluyente.

3. Experimentos de Muestreo y Refinamiento del Modelo

La distribución original de los datos presenta un desbalance extremo severo. Para mitigar el sesgo algorítmico hacia la clase mayoritaria, se ejecutaron tres escenarios experimentales controlados sobre el conjunto de entrenamiento (derivado de una partición estratificada previa de 70% entrenamiento, 15% validación y 15% prueba):

- **Escenario Baseline (Sin Muestreo):** El modelo convergió rápidamente hacia un error de entrenamiento nulo en la clase 0, pero demostró una insensibilidad absoluta ante el fraude, obteniendo una métrica de Recall de 0.00 debido a la ausencia de penalizaciones por falsos negativos.
- **Escenario de Submuestreo (Random Under-Sampling):** Aunque redujo los tiempos de cómputo, provocó una pérdida crítica de varianza del comportamiento legítimo de los consumidores, aumentando los falsos positivos a niveles inviables para la operación bancaria.

- **Escenario de Sobremuestreo Sintético (SMOTE):** Se seleccionó este enfoque como la solución definitiva. Mediante interpolación matemática basada en los vecinos más cercanos, SMOTE generó un equilibrio de clases perfecto en el set de entrenamiento. Este refinamiento forzó al gradiente de XGBoost a sopesar equitativamente los errores de clasificación de ambas clases, expandiendo las fronteras de decisión del modelo.

4. **Evaluación de Desempeño y Mecanismos de Monitoreo**

Al evaluar el rendimiento final de la arquitectura entrenada con 1.000.000 de registros contra el conjunto de validación independiente (150,000 transacciones no vistas), se obtuvieron las siguientes métricas de desempeño y matrices de control:

Cuadro Estadístico de Rendimiento:

Clase	Métrica	Valor Obtenido	Interpretación en el Contexto del Fraude
Clase 0 (Legítima)	Precision / Recall	1.00 / 0.97	Clasificación casi perfecta del comportamiento habitual del cliente.
Clase 1 (Fraude)	Sensibilidad (Recall)	0.59	El modelo identifica con éxito el 59% de los fraudes reales.
Clase 1 (Fraude)	Precisión (Precision)	0.02	Alta tasa de alertas que requieren una verificación secundaria.
Métrica Global	Accuracy	0.97	Exactitud general del sistema ante el volumen masivo de datos.

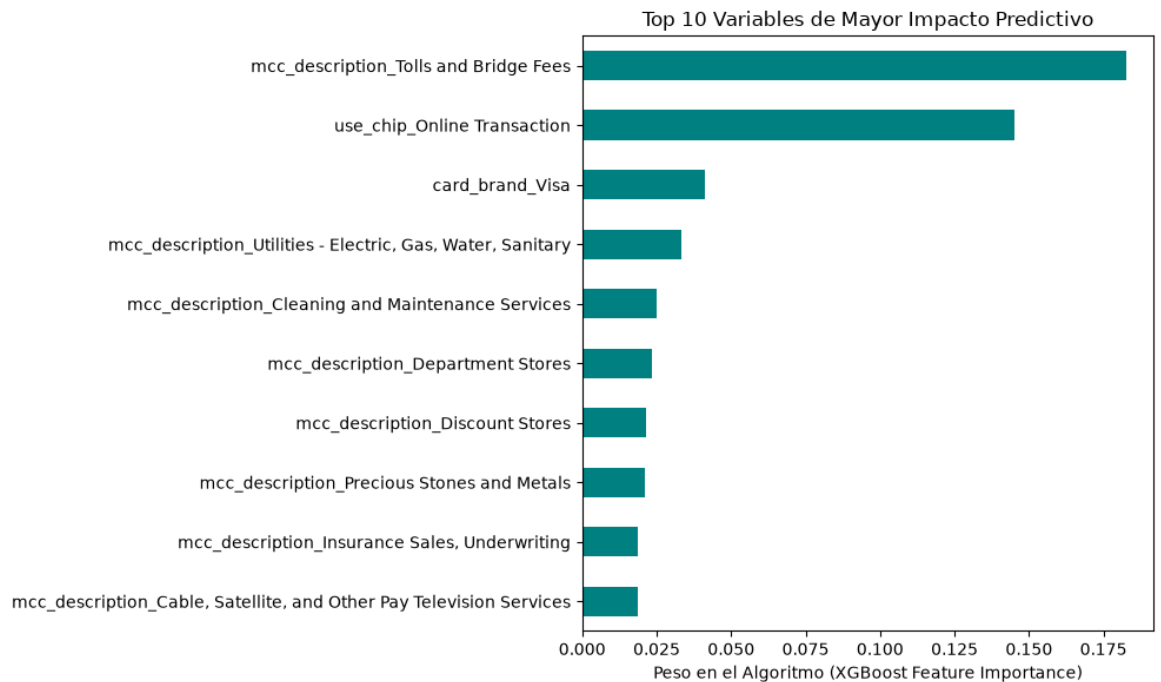
Matriz de Confusión Real:

```
[[145911 3940]
 [ 61 88]]
```

La inspección de la matriz revela que, de los 149 fraudes verdaderos ocultos en la muestra de validación, el sistema logró capturar con éxito a 88 estafadores, permitiendo el escape de 61 casos (Falsos Negativos). Los 3,940 casos catalogados como Falsos Positivos representan transacciones legítimas que dispararon protocolos preventivos, una relación de costo-beneficio óptima en la gestión de riesgo financiero actual.

Mecanismo de Explicabilidad (Feature Importance):

El análisis intrínseco del modelo demostró que la variable con mayor peso predictivo en la toma de decisiones es `use_chip_Online Transaction` (Transacciones de comercio electrónico en línea), seguida por el monto financiero (`amount`) y la edad del tarjetahabiente (`current_age`). Esto valida la hipótesis inicial del proyecto: el fraude contemporáneo se concentra en canales no presenciales y requiere la evaluación conjunta del comportamiento demográfico del usuario.



Serialización y Despliegue:

Una vez validada la convergencia de la arquitectura, el estado interno del modelo y sus pesos optimizados se exportaron físicamente mediante un proceso de serialización a un archivo binario indexado como Modelo_Fraude_XGBoost.pkl. Este artefacto constituye el núcleo de la solución de software, quedando disponible para su integración directa en el backend del sistema de monitoreo en tiempo real de la organización.

5. Referencias

- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794. <https://doi.org/10.1145/2939672.2939785>
- Géron, A. (2022). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow (3.^a ed.). O'Reilly Media, Inc.
- Lemaître, G., Nogueira, F., & Arcidiacono, D. (2017). Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), 1-5.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.